■ 3.9.2 The Uncertainties of Numerical Mathematics

Mathematica does operations like numerical integration very differently from the way it does their symbolic counterparts.

When you do a symbolic integral, *Mathematica* takes the functional form of the integrand you have given, and applies a sequence of exact symbolic transformation rules to it, to try and evaluate the integral.

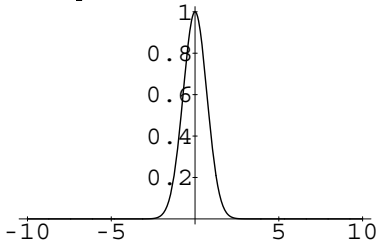
When you do a numerical integral, however, *Mathematica* does not look directly at the functional form of the integrand you have given. Instead, it simply finds a sequence of numerical values of the integrand at particular points, then takes these values and tries to deduce from them a good approximation to the integral.

An important point to realize is that when *Mathematica* does a numerical integral, the *only* information it has about your integrand is a sequence of numerical values for it. To get a definite result for the integral, *Mathematica* then effectively has to make certain assumptions about the smoothness and other properties of your integrand. If you give a sufficiently pathological integrand, these assumptions may not be valid, and as a result, *Mathematica* may simply give you the wrong answer for the integral.

This problem may occur, for example, if you try to integrate numerically a function which has a very thin spike at a particular position. *Mathematica* samples your function at a number of points, and then assumes that the function varies smoothly between these points. As a result, if none of the sample points come close to the spike, then the spike will go undetected, and its contribution to the numerical integral will not be correctly included.

Here is a plot of the function $\exp(-x^2)$.

```
In[1]:= Plot[Exp[-x^2], {x, -10, 10}, PlotRange->All]
```



`NIntegrate` gives the correct answer for the numerical integral of this function from -10 to +10.

```
In[2]:= NIntegrate[Exp[-x^2], {x, -10, 10}]
Out[2]= 1.77245
```

If, however, you ask for the integral from -10^6 to $+10^6$, `NIntegrate` will miss the peak near $x = 0$, and give the wrong answer.

```
In[3]:= NIntegrate[Exp[-x^2], {x, -10^6, 10^6}]
Out[3]= 0.
```

Although `NIntegrate` follows the principle of looking only at the numerical values of your integrand, it nevertheless tries to make the best possible use of the information that it can get. Thus, for example, if `NIntegrate` notices that your integrand is changing rapidly in a particular region, it will take more samples in that region. In this way, `NIntegrate` tries to "adapt" its operation to the particular integrand you have given.

The kind of adaptive procedure that `NIntegrate` uses is similar, at least in spirit, to what `Plot` does in trying to draw smooth curves for functions. In both cases, *Mathematica* tries to go on taking more samples in a particular region until it has effectively found a smooth approximation to the function in that region.

The kinds of problems that can appear in numerical integration can also arise in doing other numerical operations on functions.

For example, if you ask for a numerical approximation to the sum of an infinite series, *Mathematica* samples a certain number of terms in the series, and then does an extrapolation to estimate the contributions of other terms. If you insert large terms far out in the series, they may not be detected when the extrapolation is done, and the result you get for the sum may be incorrect.

A similar problem arises when you try to find a numerical approximation to the minimum of a function. *Mathematica* samples only a finite number of values, then effectively assumes that the actual function interpolates smoothly between these values. If in fact the function has a sharp dip in a particular region, then *Mathematica* may miss this dip, and you may get the wrong answer for the minimum.

If you work only with numerical values of functions, there is simply no way to avoid the kinds of problems we have been discussing. Exact symbolic computation, of course, allows you to get around these problems.

In many calculations, it is therefore worthwhile to go as far as you can symbolically, and then resort to numerical methods only at the very end. This gives you the best chance of avoiding the problems that can arise in purely numerical computations.

One might imagine that you could use symbolic methods to check for any features, say in the integrand of a numerical integral, that would give rise to problems in numerical computation. As soon as your integrand contains conditionals, control structures or nested function calls, there can be no general procedure, however, to do the tests that are needed. Nevertheless, for specific classes of integrands, it may be possible to perform some such tests. You can always implement these tests by defining special rules for `NIntegrate` in certain cases.